

AI Threat Modeling Foundations

Chapter 06 · Episode 01 · Domain 2.1 — What to Secure, and Why



In This Episode

1. Why threat modeling needs AI-specific extensions
2. The four primary AI attack surfaces
3. A structured process for AI deployments
4. Connect threat findings to control selection

Traditional vs AI Systems

TRADITIONAL

Deterministic — code does what it says

Traceable, auditable data flows

Enumerable trust boundaries

AI SYSTEMS

Same input → different outputs

Behavior emerges from training data

Flaws embedded before runtime

What Makes AI Different



DATA

Attack surface includes data, not just code



TIMING

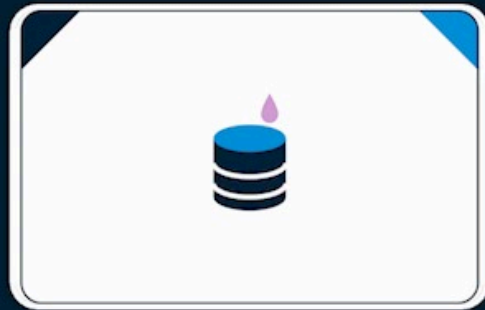
Training-time attacks surface weeks later



ASSET

The model itself carries IP value

The Four AI Attack Surfaces



DATA

Poisoning · labels ·
supply chain



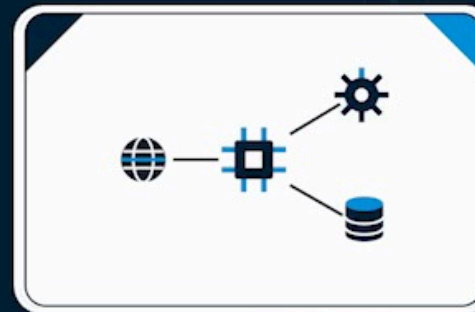
INFERENCE

Prompt injection · jailbreaking



MODEL

Inversion · extraction · theft



INTEGRATION

Excessive agency · indirect injection



Six-Step Process

1. Define the AI system scope
2. Decompose — map flows across surfaces
3. Identify threats per surface
4. Assess likelihood and impact
5. Select controls for each threat
6. Document and review — a living artifact

Invisible Trust Boundaries



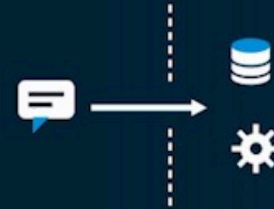
TRAINING

Model inherits any training-pipeline compromise



PROMPT

Injection turns user input into operator orders



TOOLS

Tool calls cross into real-world execution

Threats Drive Controls

Guardrails → inference interface

Access controls → model + integration

Encryption + data security → data layer

Monitoring → all four surfaces



1 6-STEP PROCESS
Walkthrough

2 4-SURFACE
Threat Catalog

3 LIKELIHOOD × IMPACT
Risk Register

4 PRIORITIZED
Control Map

HOW TO RUN

WINDOWS

```
python ai_threat_model_builder.py
```

MACOS/LINUX

```
python3 ai_threat_model_builder.py
```

TRY IT Set `inference_api='public'` and `authentication=False`. Observe how the risk register changes.

`ai_threat_model_builder.py`

pure Python 3.10+, no external libraries

Demo: AI Threat Model Builder

Six-step process across four surfaces → prioritized risk register

Episode Takeaways

Four surfaces — data · model · inference interface · integration

Six steps — scope definition through documentation

Trust boundaries are often invisible

The threat model makes every control decision defensible

Applying OWASP LLM Top 10

Chapter 06 · Episode 02 · Domain 2.1 — Ten Risks, Named and Mapped



In This Episode

1. Identify all ten categories by name
2. Map each category to its attack surface
3. Recognize categories in scenarios
4. Connect each to its primary mitigation

Web Top 10 vs LLM Top 10

WEB APPS

Vulnerabilities live in code
SQL injection — deterministic
Same string, same exploit

LLM APPS

Language · behavior · integration
Prompt injection — probabilistic
Depends on instruction weighing

A Standard Vocabulary

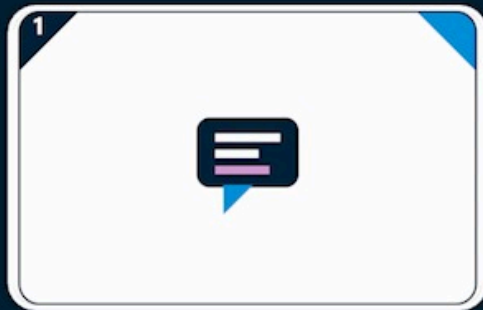
Threat model finding → OWASP name

Scenario → category · category → control

Recognize by name AND description

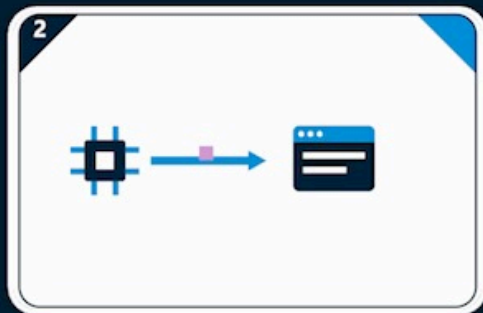


LLM01–LLM04



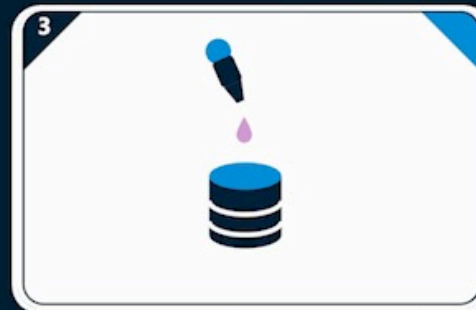
LLM01

Prompt injection —
direct + indirect



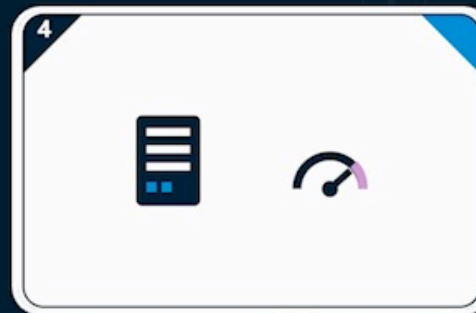
LLM02

Insecure output → XSS,
RCE downstream



LLM03

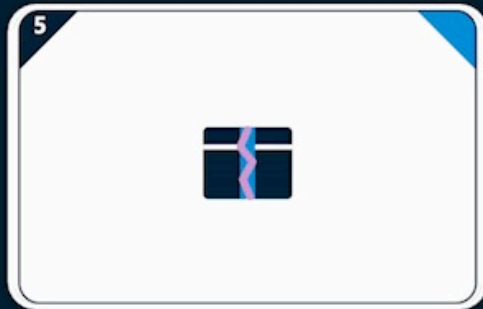
Training data
poisoning — backdoors



LLM04

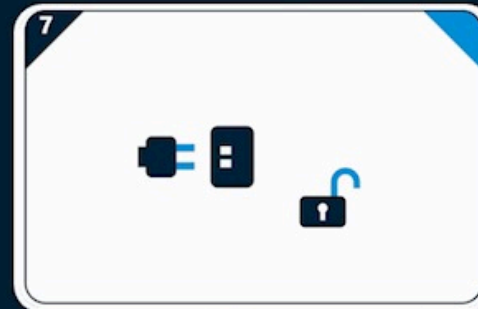
Model DoS — token flooding

LLM05–LLM08



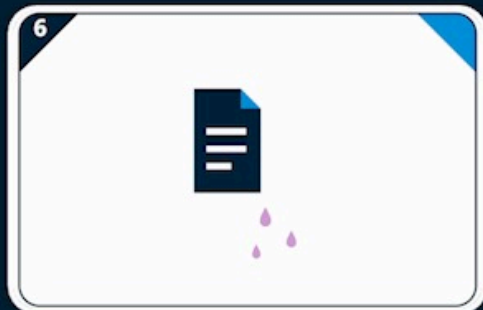
LLM05

Supply chain —
compromised components



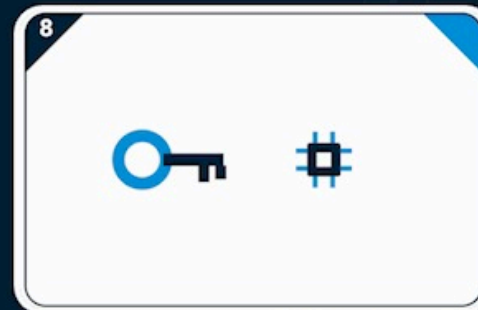
LLM07

Insecure plugin design —
no scoping



LLM06

Sensitive
information disclosure



LLM08

Excessive agency —
autonomous acts

LLM09 & LLM10

LLM09

Overreliance —
outputs unverified

LLM10

Model theft
via API probing

CONTROLS

Confidence thresholds · rate limits · auth



Ten Categories, Four Surfaces

LLM01–02 → inference interface + output

LLM03 → data layer

LLM04–05 → infrastructure + supply chain

LLM06–08 → integration

LLM09–10 → governance

The 3-Question Framework



WHEN

Training time, inference time, or post-deployment?



ASSET

Behavior, weights, data, or system integrity?



CAUSE

Missing technical control or governance gap?

1 LLM01-04
Interface & Data

2 LLM05-08
Supply Chain & Agency

3 LLM09-10
Operational

4 RISK-RATED
Findings Report

HOW TO RUN

WINDOWS

```
python owasp_llm_top10_scanner.py
```

MACOS/LINUX

```
python3 owasp_llm_top10_scanner.py
```

TRY IT Set `agent_mode=True` and `plugin_validation=False`. Observe LLM07 and LLM08 risk levels increase.

`owasp_llm_top10_scanner.py`

evaluates deployment against all 10 categories

Demo: LLM Top 10 Scanner

Scan a deployment against all ten — then flip `agent_mode=True`

Know the Numbers Cold

A shared vocabulary for LLM risk

Category numbers appear in questions

Memorize LLM01–LLM10



MITRE ATLAS and Adversarial Techniques

Chapter 06 · Episode 03 · Domain 2.1 — How Adversaries Attack AI

ATT&CK vs ATLAS

MITRE ATT&CK

Tactics: recon → impact

Exploits software + config flaws

Deterministic — same exploit each time

MITRE ATLAS

Same tactic-technique hierarchy

Exploits ML mathematical properties

Success depends on model behavior

Complementary Frameworks

Some techniques have no ATT&CK equivalent
OWASP names the vulnerability
ATLAS describes the technique
Both appear on the exam

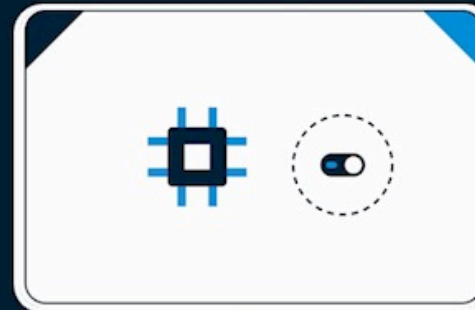


Data Layer — Before Deployment



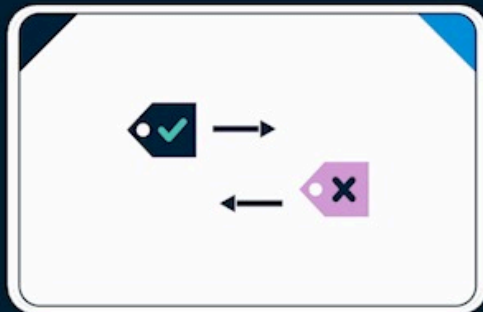
POISONING

Malicious samples corrupt training



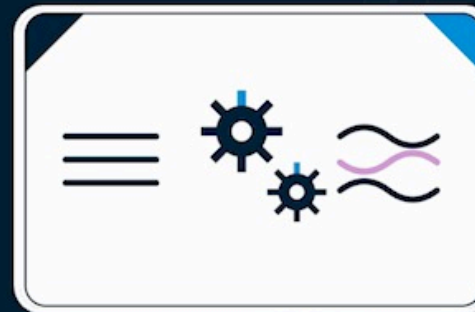
BACKDOOR

Trigger-only behavior — passes tests



LABELS

Mislabeled data shifts boundaries



FEATURES

Preprocessing logic altered

Model Layer Techniques

INVERSION

Reconstruct training data

MEMBERSHIP

Was this record
in training?

EXTRACTION

Substitute model
via API queries

ADVERSARIAL

Perturbed inputs
evade classifiers

TARGET

Trained weights — IP and privacy risk

MODEL INVERSION

QUERY 1

QUERY 2

QUERY N



 **THE MODEL** CONFIDENCE SCORES OUT

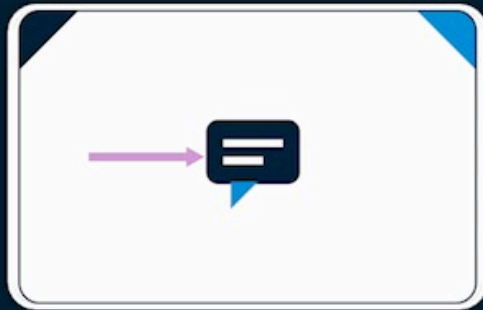


TRAINING DATA

RECONSTRUCTED

Enough crafted queries pull private records back out. Rate limiting is the primary control.

Interface & Integration Layer



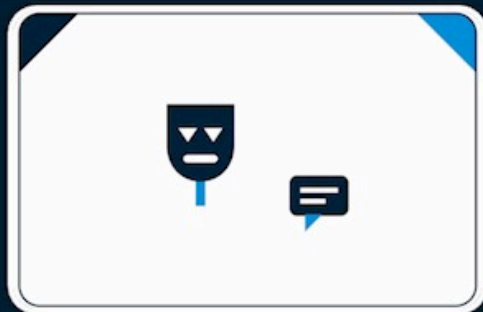
INJECTION

Direct + indirect
instruction override



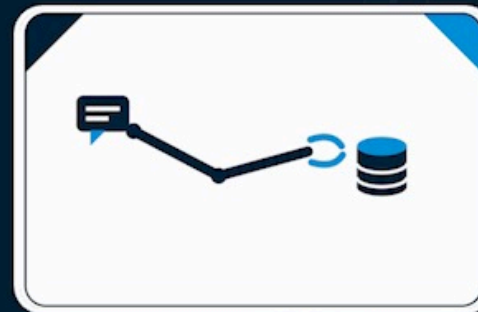
API ABUSE

Token flooding · rate exploitation



JAILBREAK

Roleplay + spoofing
remove guardrails



PRIVILEGE

Tool calls escalate into systems

Attack Chain Mapping

Recon → extract → analyze → evade

Supply chain → backdoor → trigger fire

Control gaps live at stage transitions

ONE ATTACK, FOUR STAGES

1 · RECON

PUBLIC API PROBING



2 · EXTRACTION

QUERY HARVESTING



UNDETECTED

3 · OFFLINE ANALYSIS

OUT OF YOUR SIGHT



4 · EVASION

CRAFTED INPUTS

Trace the full chain — the hop you cannot see is the one to instrument.

1 CHAIN 1
Extraction → Evasion

2 CHAIN 2
Backdoor Activation

3 CONTROL TOGGLE
See gaps close

4 DEFENSE MAP
Per stage

HOW TO RUN

WINDOWS

```
python atlas_attack_chain_mapper.py
```

MACOS/LINUX

```
python3 atlas_attack_chain_mapper.py
```

TRY IT Set `CONTROLS_ENABLED['rate_limiting']=False`.
Watch Chain 1 advance to Stage 3 before
being stopped.

`atlas_attack_chain_mapper.py`

simulates two attack chains, maps ATLAS techniques, shows where
controls break the chain

Demo: ATLAS Chain Mapper

Simulate both chains — add rate limiting, watch the chain break

Episode Takeaways

ATLAS = ATT&CK structure applied to ML systems

Data layer attacks embed before deployment

Model layer targets weights, IP, and privacy

Chains sequence — gaps live at transitions

Building an AI Threat Model

Chapter 06 · Episode 04 · Domain 2.1 — From Description to Risk Register



In This Episode

1. Produce a threat model from a system description
2. Risk statements in condition–consequence–asset form
3. Prioritize with risk appetite and thresholds
4. Connect findings to Domain 2 controls

Describe the System First

FUNCTION

What decisions?
Wrong-decision cost?

DATA

Training · inputs ·
stores · outputs

INTEGRATIONS

Connections + tool
permission scope

DEPLOYMENT

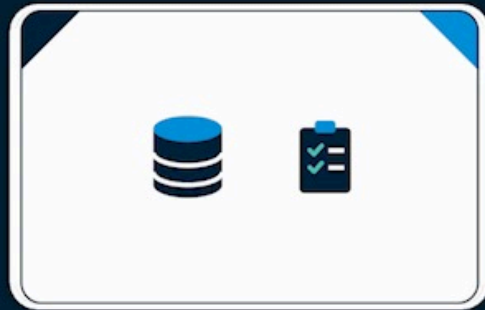
Who reaches the
interface? Auth?

FAILURE

Worst case sets the impact ceiling



Walk the Four Surfaces



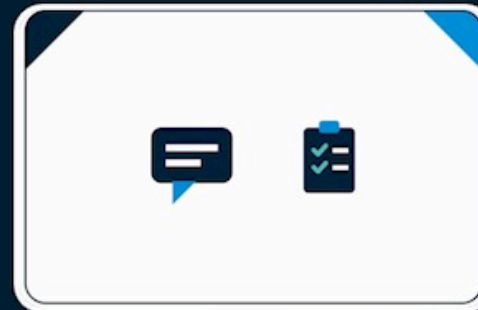
DATA

External sources?
RAG? → LLM03, LLM01



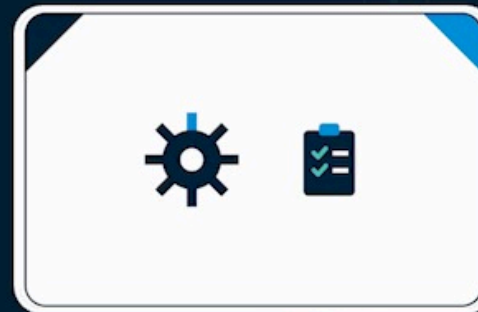
MODEL

Public API? PII?
→ LLM10, LLM06



INTERFACE

Free text? Prompts?
→ LLM01, LLM09



INTEGRATION

Tool calls? Writes?
→ LLM08, T0052

If **CONDITION** → **CONSEQUENCE** → **ASSET**

Vague threats can't be prioritized

Risk statements make lists actionable

RISK STATEMENT FORMAT

IF

CONDITION



THEN

CONSEQUENCE

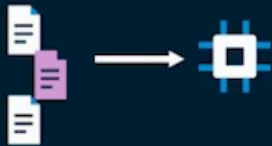


AFFECTING

ASSET

Write every threat in the model this way.

Risk Statements in Practice



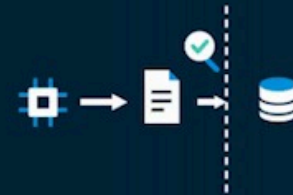
DATA

Unvalidated intel feeds →
poisoned triage



INTERFACE

No injection detection →
prompt override



INTEGRATION

Unapproved SIEM writes →
escalation risk

Score Every Threat

LIKELIHOOD

Exposure — public or internal API?

Auth + guardrails in place?

Attacker sophistication required

IMPACT

Confidentiality · integrity

Availability · regulatory

Reputational — public trust

Appetite → Thresholds

Human approval before write operations

Below 80% confidence → human analyst

F1 drift over 5% → pause automation

Quarterly review + after every change



Documentation Artifacts



REGISTER

Threats scored,
owned, reviewed



DATASET

Sources · licensing · known bias



MODEL CARD

Function · limits ·
failure modes



CHANGE LOG

Retraining + integration history

Episode Takeaways

Description quality determines model quality

Walk four surfaces with OWASP + ATLAS checklists

Risk format: condition → consequence → asset

Documentation artifacts are governance evidence